



Convolution on RGB Images



1. Understanding RGB Images

An **RGB image** is composed of three color channels: **Red, Green, and Blue**. Each channel is a 2D matrix of pixel intensity values (typically ranging from 0 to 255). These three matrices are stacked together to form a full-color image.

- **Image Shape:** (Height × Width × 3)
 - The 3 represents the three color channels (R, G, B).

Example Breakdown:

- **Red Channel:**

$$\begin{bmatrix} R_{11} & R_{12} & \dots & R_{1W} \\ R_{21} & R_{22} & \dots & R_{2W} \\ \vdots & \vdots & \ddots & \vdots \\ R_{H1} & R_{H2} & \dots & R_{HW} \end{bmatrix}$$

- **Green Channel:**

$$\begin{bmatrix} G_{11} & G_{12} & \dots & G_{1W} \\ G_{21} & G_{22} & \dots & G_{2W} \\ \vdots & \vdots & \ddots & \vdots \\ G_{H1} & G_{H2} & \dots & G_{HW} \end{bmatrix}$$

- **Blue Channel:**

$$\begin{bmatrix} B_{11} & B_{12} & \dots & B_{1W} \\ B_{21} & B_{22} & \dots & B_{2W} \\ \vdots & \vdots & \ddots & \vdots \\ B_{H1} & B_{H2} & \dots & B_{HW} \end{bmatrix}$$

2. Convolution Process on RGB Images

Step 1: Applying Filters (Kernels)

- A filter (or kernel) is a small matrix used for feature extraction.
- For RGB images, the **filter has the same depth as the input**, i.e., 3 channels.
- A **3×3 filter** will have shape **(3×3×3)** for RGB.

Filter Example (3×3×3):

RGB Filter Weights:

$$\begin{bmatrix} K_{R11} & K_{R12} & K_{R13} \\ K_{R21} & K_{R22} & K_{R23} \\ K_{R31} & K_{R32} & K_{R33} \end{bmatrix}, \begin{bmatrix} K_{G11} & K_{G12} & K_{G13} \\ K_{G21} & K_{G22} & K_{G23} \\ K_{G31} & K_{G32} & K_{G33} \end{bmatrix}, \begin{bmatrix} K_{B11} & K_{B12} & K_{B13} \\ K_{B21} & K_{B22} & K_{B23} \\ K_{B31} & K_{B32} & K_{B33} \end{bmatrix}$$

- The filter slides over each spatial region of the image.
- For each region, it performs **element-wise multiplication** across all three channels and **sums the result**.

Step 2: Summing Across Channels

- After convolution over each channel (R, G, B), the results are summed:

$$\text{Output}(i, j) = \text{Conv}_R(i, j) + \text{Conv}_G(i, j) + \text{Conv}_B(i, j)$$

Step 3: Repeat for All Filters

- CNN layers typically have **multiple filters** (e.g., 32, 64).
- The process above is repeated for **each filter**.
- The result is a **stack of feature maps** — one for each filter.

Mathematical Representation

Let:

- Input image shape: $(H \times W \times 3)$
- Filter shape: $(F \times F \times 3)$
- Output pixel at location (i, j) :

$$\text{Output}(i, j) = \sum_{c=1}^3 \sum_{m=1}^F \sum_{n=1}^F \text{Image}(i + m - 1, j + n - 1, c) \cdot \text{Filter}(m, n, c)$$

Where:

- (i, j) : spatial indices
 - (c) : channel index (R, G, B)
 - (m, n) : are the indices of the filter window
-

Stride and Padding

- **Stride (S)**: The step size with which the filter moves across the image.
 - Stride = 1 → moves one pixel at a time.
 - Stride = 2 → skips one pixel, producing smaller outputs.
 - **Padding (P)**: Adding zeros around the image border to preserve spatial dimensions or control output size.
-

Output Feature Maps

- **Each filter** produces **one 2D feature map** after convolution.
- If we use **K filters**, the output shape becomes:
(H_{out} × W_{out} × K)

Where:

$$H_{out} = \frac{H - F + 2P}{S} + 1, \quad W_{out} = \frac{W - F + 2P}{S} + 1$$

- **Example:**
Input = (128 × 128 × 3)
Filters = 32 filters of size 3×3×3, Stride = 1, Padding = 1
→ Output = (128 × 128 × 32)
-

Summary

- RGB images have 3 channels: Red, Green, Blue.
- Filters in CNN must match the input depth (3 for RGB).
- Each filter processes all three channels and outputs a single 2D map.
- Multiple filters = multiple feature maps stacked together.
- Stride and padding affect the spatial size of the output.

Global Average Pooling (GAP) and Gaussian Blur

Global Average Pooling (GAP)

GAP operates over the entire spatial dimensions of a feature map. It computes the **average of all the values in each feature map**, reducing it to a **single scalar** per map.

Purpose of GAP

1. Dimensionality Reduction

- GAP converts each 2D feature map into a single scalar.
- If input is $(H \times W \times C)$, output becomes $(1 \times 1 \times C)$, i.e., a 1D vector of size C .

2. Simplification

- Eliminates the need for large fully connected layers.
- Reduces parameters, helps in preventing overfitting, and speeds up computation.

3. Global Feature Representation

- Focuses on *presence* of a feature rather than *location*.
 - Useful when spatial information is less important.
-

Example of Global Average Pooling

Given a feature map of size $(8 \times 8 \times 32)$:

- 8×8 = spatial size (height $\times$ width)
- 32 = number of feature maps (channels)

Let's simplify with a 3×3 example:

Feature Map:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Global Average Pooling (GAP) Output:

$$\frac{1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9}{9} = \frac{45}{9} = 5$$

If we had 32 such feature maps, the output would be a vector:

GAP Output: $[\text{avg}_1, \text{avg}_2, \dots, \text{avg}_{32}]$

→ Shape: $(1 \times 1 \times 32)$

When to Use GAP

1. Replacing Fully Connected Layers

- Used in architectures like **Inception** and **ResNet**.

2. Object Localization

- Encourages the model to detect features regardless of their exact position.

3. Preventing Overfitting

- Reduces parameter count by eliminating dense layers.

4. Global Feature Representation

- Summarizes the presence of a feature globally across the entire map.

GAP vs Global Max Pooling

Method	What it Does	Focus
Global Average Pooling	Averages all values in the feature map	General feature presence
Global Max Pooling	Takes the max value in the feature map	Most prominent feature

Gaussian Blur

Gaussian Blur is a smoothing technique used to reduce **noise** and **detail** in images. It's widely used in **preprocessing** steps for tasks like edge detection or denoising.

Concept and Purpose

1. Smoothing

- Averages out local intensities.
- Reduces random noise.

2. Detail Reduction

- Blurs fine edges.
 - Helps suppress unnecessary features before edge detection.
-

Gaussian Function

The blurring is based on the 2D Gaussian function:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

- $((x, y))$: pixel coordinates relative to the kernel center
 - (σ) : standard deviation, controls spread/blur level
-

Gaussian Kernel Example (5×5, $\sigma=1$)

A discrete 5×5 Gaussian kernel might look like this (simplified structure):

$$\begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix}$$

- Normalize by dividing by the sum of all elements.
-

Convolution Process

1. Kernel slides over the image.
 2. At each position, a **weighted sum** of neighboring pixels is calculated.
 3. This sum replaces the center pixel.
 4. Repeat for all pixels.
-

Effect of Kernel Size and Sigma

- **Larger Kernel** → more blurring
 - **Higher Sigma (σ)** → wider Gaussian curve, stronger blur
 - **Smaller Sigma** → sharper blur (less smoothing)
-

Applications of Gaussian Blur

1. **Preprocessing for Edge Detection**
 - Applied before Sobel or Canny to suppress noise-induced edges.
 2. **Image Denoising**
 - Smooths out high-frequency random noise.
 3. **Bokeh Effect Simulation**
 - Used to blur background and focus on the subject.
 4. **Downsampling Preparation**
 - Applied before reducing image resolution to prevent aliasing.
-

Keras ImageDataGenerator – Real-Time Image Augmentation in Deep Learning

What is ImageDataGenerator ?

The **Keras ImageDataGenerator** is a utility that generates batches of tensor image data **with real-time data augmentation**. It is widely used in training deep learning models, especially when we want to artificially expand the training dataset through transformations like flipping, rotating, shifting, etc.

Key Features

1. Data Augmentation

- Random transformations like **rotation, shift, flip, shear, zoom**, etc., are applied to input images.
- Helps the model generalize better by exposing it to various distorted versions of the same input image.

2. Real-Time Generation

- Images are **augmented on the fly** during training.
- Saves memory by **not pre-generating or storing** augmented images.

3. Normalization & Preprocessing

- Supports **rescaling pixel values** (e.g., from [0, 255] to [0, 1]).
 - Can also apply custom preprocessing functions to ensure optimal input for training.
-

Common Parameters of ImageDataGenerator

Parameter	Description
<code>rotation_range</code>	Degree range for random rotations
<code>width_shift_range</code>	Fraction of total width for horizontal shifts
<code>height_shift_range</code>	Fraction of total height for vertical shifts
<code>shear_range</code>	Shear angle in counter-clockwise direction (in degrees)
<code>zoom_range</code>	Range for random zoom
<code>horizontal_flip</code>	Randomly flips the image horizontally
<code>vertical_flip</code>	Randomly flips the image vertically
<code>rescale</code>	Rescales the pixel values by a given factor (e.g., <code>1/255</code>)
<code>fill_mode</code>	Strategy to fill pixels outside boundaries (options: <code>'nearest'</code> , <code>'constant'</code> , <code>'reflect'</code> , <code>'wrap'</code>)

Output Shape Example

Suppose the output shape from a convolutional layer is:

Output Shape: ((None, 148, 148, 32))

Breakdown:

- `None` → **Batch size**, which is dynamic and determined at runtime.
- `148, 148` → **Height and Width** of the feature map.
- `32` → **Number of channels** (i.e., filters or feature maps).

✓ The `None` makes the model flexible enough to handle **any batch size** during training or inference (e.g., 32, 64, 128...).

Why Use `None` in Output Shape?

Using `None` as the batch size:

- Makes the model **flexible and adaptable** to different batch sizes.
 - Ensures compatibility in **varied training/inference environments**.
 - During actual training, `None` is **replaced by the batch size** you specify.
-

Summary

Keras `ImageDataGenerator` is essential for:

- Efficient **real-time image augmentation**
- **Reducing overfitting** by increasing dataset diversity
- Providing a **scalable and memory-efficient** pipeline for image input

Use it whenever your model trains on **limited image data** or when you need to **boost generalization!**

Callbacks in Keras

What are Callbacks?

Callbacks in Keras are special functions or classes that are executed **at specific stages** during training (like end of epoch, end of batch, etc.).

They allow you to:

- **Monitor training**
- **Adjust learning rates**
- **Save the model**
- **Stop training early**
- **Log metrics for visualization**

They extend the training loop and are **essential for optimizing, controlling, and debugging** deep learning models.

Why Use Callbacks?

Purpose	Description
Monitor & Control Training	React to metrics like loss/accuracy to modify behavior.
Improve Performance	Dynamically adjust hyperparameters like learning rate.
Save Models	Automatically save model weights at specific checkpoints.
Prevent Overfitting	Use EarlyStopping to halt training when validation metrics stagnate.
Log Metrics	Save logs for tools like TensorBoard or CSV for later analysis.

Common Use Cases

1. Early Stopping

- Stop training when validation loss stops improving.

2. Model Checkpointing

- Save the model when validation performance improves.

3. Learning Rate Scheduling

- Reduce learning rate when the model hits a performance plateau.

4. TensorBoard Visualization

- Log metrics, graphs, histograms, etc.

5. CSV Logging

- Save training logs in `.csv` for manual inspection.
-

Popular Callbacks in Keras

1. EarlyStopping

```
from tensorflow.keras.callbacks import EarlyStopping
```

```
early_stopping = EarlyStopping(  
    monitor='val_loss',  
    patience=5,  
    restore_best_weights=True  
)
```

- **monitor:** Metric to monitor (e.g., 'val_loss', 'val_accuracy')
- **patience:** Number of epochs with no improvement after which training will be stopped
- **restore_best_weights:** If True, restores model weights from the epoch with the best monitored value

2. ModelCheckpoint

```
from tensorflow.keras.callbacks import ModelCheckpoint
```

```
checkpoint = ModelCheckpoint(  
    filepath='model.h5',  
    monitor='val_loss',  
    save_best_only=True  
)
```

- **filepath:** Path where the model is saved (e.g., 'model.h5')
- **monitor:** Metric to evaluate for saving the model (e.g., 'val_loss')
- **save_best_only:** If True, saves the model only when the monitored metric improves

3. ReduceLROnPlateau

```
from tensorflow.keras.callbacks import ReduceLROnPlateau
```

```
reduce_lr = ReduceLROnPlateau(  
    monitor='val_loss',  
    factor=0.1,  
    patience=10  
)
```

- **monitor:** Metric to watch for stagnation (e.g., 'val_loss')
- **factor:** Multiply current learning rate by this factor (e.g., 0.1 → reduces LR by 10×)
- **patience:** Number of epochs with no improvement before reducing the learning rate

✓ Useful when training plateaus — helps the model escape local minima by adjusting the learning rate.

4. TensorBoard

```
from tensorflow.keras.callbacks import TensorBoard
```

```
tensorboard = TensorBoard(log_dir='./logs')
```

- **log_dir**: Folder where logs will be saved (e.g., `'./logs'`)

✓ Used to visualize training metrics like **loss, accuracy, learning rate**, etc.

📊 View the dashboard by running:

```
%tensorboard --logdir logs/
```

5. 📁 CSVLogger

```
from tensorflow.keras.callbacks import CSVLogger
```

```
csv_logger = CSVLogger('training_log.csv')
```

- **CSVLogger** logs all epoch-wise metrics (like **loss, accuracy**, etc.) into a `.csv` file
- File used here: `'training_log.csv'`

📊 Useful for:

- Post-training analysis
- Plotting training/validation graphs
- Debugging model performance trends

🔧 How to Use Callbacks in `model.fit`

```
model.fit(  
    X_train, y_train,  
    epochs=50,  
    validation_data=(X_val, y_val),  
    callbacks=[early_stopping, checkpoint, reduce_lr, tensorboard]  
)
```

- Pass a list of callback objects to the `callbacks` argument
- These callbacks will automatically activate during training based on their configuration

✓ Helps with:

- Early stopping
- Saving the best model
- Learning rate adjustments
- Logging and visualization

🧱 Understanding Plateaus

A **plateau** is a period during training where the model's performance metric (e.g., `val_loss`) stops improving over several epochs.

🌟 **To handle plateaus:**

- Use `ReduceLRonPlateau` to lower the learning rate automatically
- Modify the model architecture (e.g., add layers, change activation functions)
- Add **regularization** or **dropout** to prevent overfitting and improve generalization

✓ Summary

Keras callbacks help you:

- ✓ Automate training control
- 💾 Save checkpoints
- 🚫 Prevent overfitting
- 📊 Visualize performance
- 📉 Improve convergence

Convolution on RGB Images

```
In [1]: import numpy as np

# Define a simple 5x5x3 RGB image (3 channels)
image = np.array([
    [[1, 0, 2], [2, 1, 1], [3, 2, 0], [0, 1, 1], [1, 0, 2]],
    [[0, 1, 0], [1, 0, 1], [2, 2, 2], [3, 1, 3], [2, 0, 1]],
    [[3, 0, 2], [0, 1, 0], [1, 0, 1], [2, 2, 2], [1, 0, 0]],
    [[2, 1, 1], [1, 0, 2], [3, 3, 1], [0, 1, 0], [0, 2, 1]],
    [[1, 2, 2], [2, 1, 0], [0, 0, 1], [1, 2, 2], [2, 1, 1]]
])

image.shape
```

Out[1]: (5, 5, 3)

```
In [2]: # Define a 3x3x3 filter (kernel) for each channel (RGB)
kernel = np.array([
    [[0, 1, 0], [1, -1, 1], [0, 1, 0]],
    [[1, 0, 1], [0, -1, 0], [1, 0, 1]],
    [[0, 1, 0], [1, 1, 1], [0, 1, 0]]
])

kernel.shape
```

Out[2]: (3, 3, 3)

```
In [3]: image_h, image_w, image_c = image.shape

print(image_h, image_w, image_c)
kernel_h, kernel_w, kernel_c = kernel.shape
print(kernel_h, kernel_w, kernel_c )
output_h = image_h - kernel_h + 1
output_w = image_w - kernel_w + 1
```



```
output = np.zeros((output_h, output_w, 1))
output.shape
```

```
5 5 3
```

```
3 3 3
```

```
Out[3]: (3, 3, 1)
```

```
In [4]: for k in range(image_c): # Apply the convolution for each channel
        for i in range(output_h):
            for j in range(output_w):
                # print(np.sum(image[i:i+kernel_h, j:j+kernel_w, k] * kernel))
                output[i, j] = np.sum(image[i:i+kernel_h, j:j+kernel_w, k] * kernel)
                # break
            # break
        # break
output
```

```
Out[4]: array([[10.],
               [12.],
               [12.]],

              [[16.],
               [15.],
               [11.]],

              [[ 8.],
               [11.],
               [15.]])
```

```
In [5]: import numpy as np
```

```
# Define a simple 5x5x3 RGB image (3 channels)
image = np.array([
    [[1, 0, 2], [2, 1, 1], [3, 2, 0], [0, 1, 1], [1, 0, 2]],
    [[0, 1, 0], [1, 0, 1], [2, 2, 2], [3, 1, 3], [2, 0, 1]],
    [[3, 0, 2], [0, 1, 0], [1, 0, 1], [2, 2, 2], [1, 0, 0]],
    [[2, 1, 1], [1, 0, 2], [3, 3, 1], [0, 1, 0], [0, 2, 1]],
    [[1, 2, 2], [2, 1, 0], [0, 0, 1], [1, 2, 2], [2, 1, 1]]
])
```

```
# Define a 3x3x3 filter (kernel) for each channel (RGB)
kernel = np.array([
    [[0, 1, 0], [1, -1, 1], [0, 1, 0]],
    [[1, 0, 1], [0, -1, 0], [1, 0, 1]],
    [[0, 1, 0], [1, 1, 1], [0, 1, 0]]
])
```

```
# Convolution operation
```

```
def convolve_rgb(image, kernel):
    image_h, image_w, image_c = image.shape
    kernel_h, kernel_w, kernel_c = kernel.shape
    output_h = image_h - kernel_h + 1
    output_w = image_w - kernel_w + 1
    output = np.zeros((output_h, output_w, 1))

    for k in range(image_c): # Apply the convolution for each channel
```

```

        for i in range(output_h):
            for j in range(output_w):
                output[i, j] = np.sum(image[i:i+kernel_h, j:j+kernel_w, k] * kernel)
    return output

# Apply the convolution
output = convolve_rgb(image, kernel)
print("Convolution Output:\n", output)

```

Convolution Output:

```

[[[10.]
  [12.]
  [12.]]

 [[16.]
  [15.]
  [11.]]

 [[ 8.]
  [11.]
  [15.]]]

```

Implementing the same with Keras

```

In [6]: import tensorflow as tf
        from tensorflow.keras import layers, models

        # Example input shape for a 32x32 RGB image
        input_shape = (100, 100, 3)

        # Define the model
        model = models.Sequential()

        # Add Input Layer
        model.add(layers.Input(shape=input_shape))

        # Adding Convolution Layer
        model.add(layers.Conv2D(16, (3, 3), padding='same', strides=1, activation='relu'))

        # Adding Max Pooling Layer
        model.add(layers.MaxPooling2D(pool_size=(2, 2), strides=2))

        # Adding more convolutional and pooling layers as needed
        model.add(layers.Conv2D(32, (3, 3), padding='same', strides=1, activation='relu'))
        model.add(layers.MaxPooling2D(pool_size=(2, 2), strides=2))

        # Print model summary to see the structure
        model.summary()

```

Model: "sequential"

Layer (type)	Output Shape	
conv2d (Conv2D)	(None, 100, 100, 16)	
max_pooling2d (MaxPooling2D)	(None, 50, 50, 16)	
conv2d_1 (Conv2D)	(None, 50, 50, 32)	
max_pooling2d_1 (MaxPooling2D)	(None, 25, 25, 32)	



Total params: 5,088 (19.88 KB)

Trainable params: 5,088 (19.88 KB)

Non-trainable params: 0 (0.00 B)

In []: